

Lecture 10: 딥강화학습 (교재 Chapter 8 딥강화학습)

<기계학습 개론> 2019 강의
서울대학교 컴퓨터공학부
장 병 탁

교재: 장교수의 딥러닝, 홍릉과학출판사, 2017.

Biointelligence Laboratory
School of Computer Science and Engineering
Seoul National University



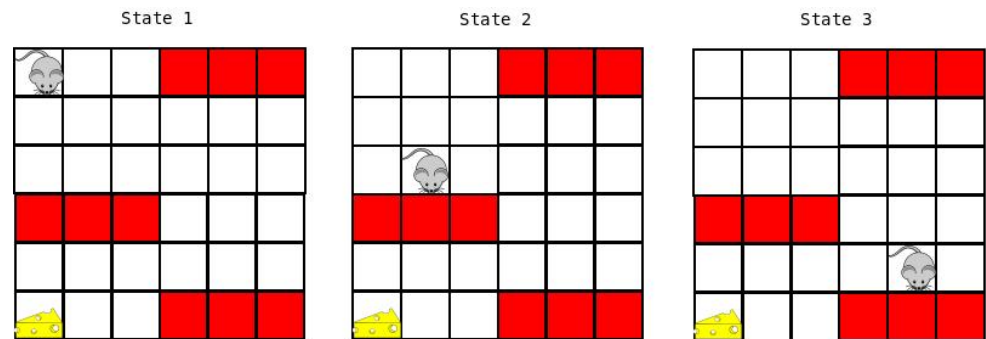
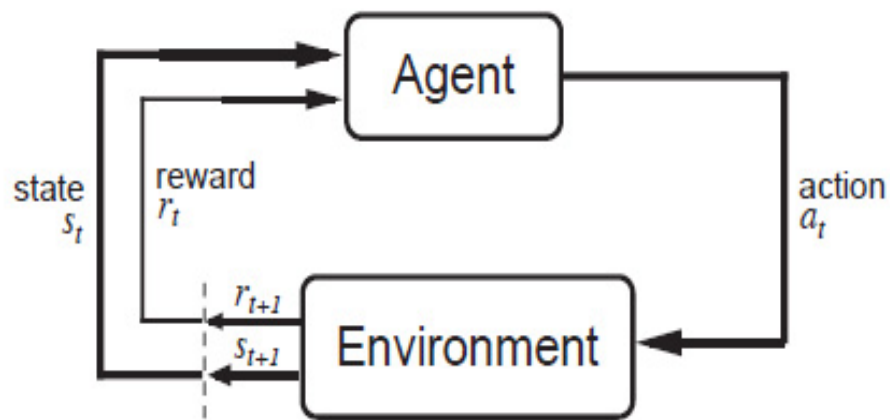
Version 20171109/20191030

목차

8.1 강화학습과 MDP 문제	3
8.2 MC 학습과 TD 학습	8
8.3 Sarsa와 Q학습 알고리즘	11
8.4 딥큐넷(DQN),.....	14
8.5 딥강화학습의 활용: AlphaGo	17
요약	19

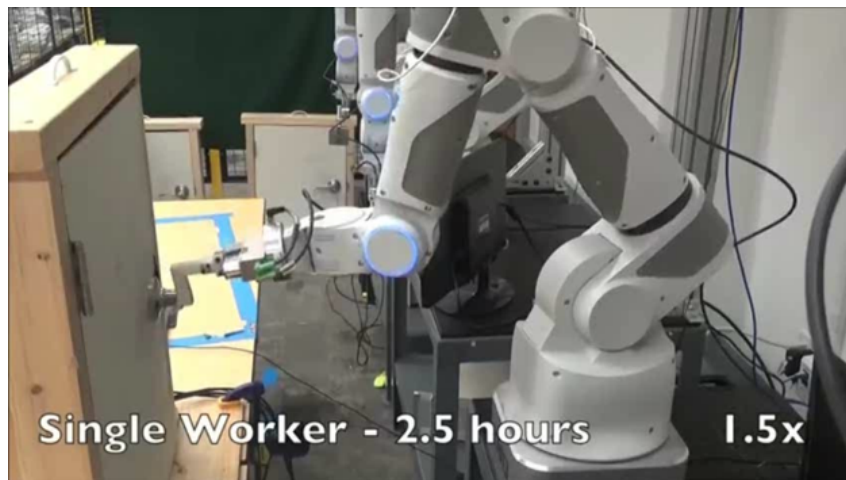
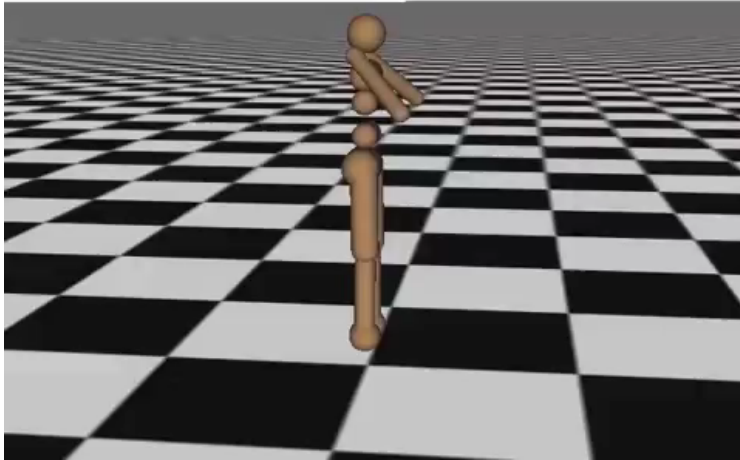
8.1 강화학습과 MDP 문제 (1/5)

- 감독학습: $y = f(x)$
 - 무감독학습: $x = f(x)$ or $x \sim p(x)$
 - 강화학습 (RL): $p(a|s)$
- 강화학습(reinforcement learning, RL)의 특징
- 에이전트의 상태(s)와 행동(a)에 대해 보상(r)을 최대화하는 정책 $p(a|s)$ 를 찾는 문제
 - 환경과 상호작용하는 목표를 가진 에이전트(로봇)
 - 순차적 의사 결정 문제, 행동 제어 문제
 - 미래 보상 고려, 지연된 보상(delayed reward)



8.1 강화학습과 MDP 문제 (2/5)

Iteration 0



8.1 강화학습과 MDP 문제 (3/5)

■ Markov Decision Process (MDP)

- Markov Decision Process = $\{S, A, P, R, \gamma\}$

S : States of the agent

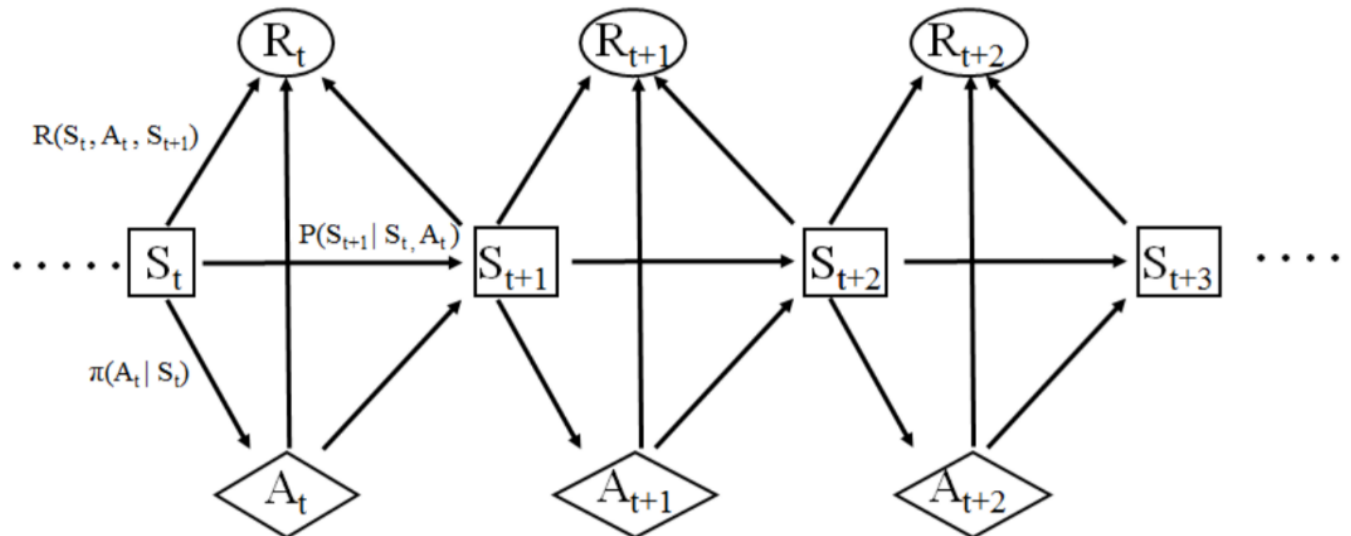
A : Actions of the agent

P : State transition probability $P_{ss'}^a = P(S_{t+1} = s' | S_t = s, A_t = a)$

R : Reward $R_s^a = E(R_{t+1} | S_t = s, A_t = a)$

γ : Discount factor

- Markov Property: $P(S_{t+1} | S_t) = P(S_{t+1} | S_{1,...,t})$



8.1 강화학습과 MDP 문제 (4/5)

- 강화학습 (RL) = Approximate Solution to MDP
- 반환 **G** (return): Discounted accumulated mean reward

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1}$$

- 정책 π (policy of the agent):

$$\pi(a | s) = P(A_t = a | S_t = s)$$

- 가치함수 **V** (value function = 장기적 반환):

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s] = E_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right]$$

$$Q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a] = E_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a\right]$$

8.1 강화학습과 MDP 문제 (5/5)

- **최적 정책:** Value function을 최대화하는 정책을 찾음

$$\pi^* = \arg \max_{\pi} V_{\pi}(s)$$

$$\pi^* = \arg \max_{\pi} Q_{\pi}(s, a)$$

- **벨만 최적식 (Bellman optimality equation)**

$$V^*(s) = \max_a \left[R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V^*(s') \right]$$

$$Q^*(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \max_{a'} Q^*(s', a')$$

- 동적 프로그래밍(dynamic programming, DP)

8.2 MC 학습과 TD 학습 (1/3)

■ 강화학습의 학습 패러다임

- Evaluation: $Q_\pi(s, a)$ 를 현재의 정책 $\pi(s)$ 를 통해 학습
- Improvement: $\pi(s)$ 를 현재의 가치함수 $Q_\pi(s, a)$ 를 통해 학습

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} q_*$$

- Improvement의 예: Greedy improvement

$$\pi_{k+1}(s) = \arg \max_a Q_k(s, a)$$

8.2 MC 학습과 TD 학습 (2/3)

■ Monte-Carlo RL (MC 학습)

- DP 방법에서 에피소드의 Return (G)을 Value function으로 부터 Sampling하여 근사(MC 근사)

$$G_t = [R_{t+1} + \gamma R_{t+2} + \dots | S_t = s, A_t = a]$$

$$q_{\pi}(s, a) = E[G_1(s, a) + G_2(s, a) + \dots | S_t = s, A_t = a]$$

- 에피소드가 끝나야 학습이 가능
- 에피소드의 길이가 길거나 무한한 경우 사용하기 어려움

Initialize:

$\pi \leftarrow$ policy to be evaluated

$V \leftarrow$ an arbitrary state-value function

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:

Generate an episode using π

For each state s appearing in the episode:

$G \leftarrow$ return following the first occurrence of s

Append G to $Returns(s)$

$V(s) \leftarrow \text{average}(Returns(s))$

8.2 MC 학습과 TD 학습 (3/3)

■ Temporal Difference RL (TD 학습)

$$V_{\pi}(s) = E[R_{t+1} + \gamma V_{\pi}(s_{t+1}) | S_t = s]$$

$$Q_{\pi}(s, a) = E[R_{t+1} + \gamma Q_{\pi}(s_{t+1}, a_{t+1}) | S_t = s, A_t = a]$$

- Bootstrapping을 사용해 에피소드가 끝나지 않아도 학습 가능
- MC 방법에 비해 효율적

□ 모델기반 vs. 모델프리 RL

- MC: 모델기반 (전이확률 필요)
- TD: 모델프리(전이확률 불필요)

□ 오프라인 vs. 온라인 학습

- MC: 오프라인(샘플 모아서 update)
- TD: 온라인(바로 update)

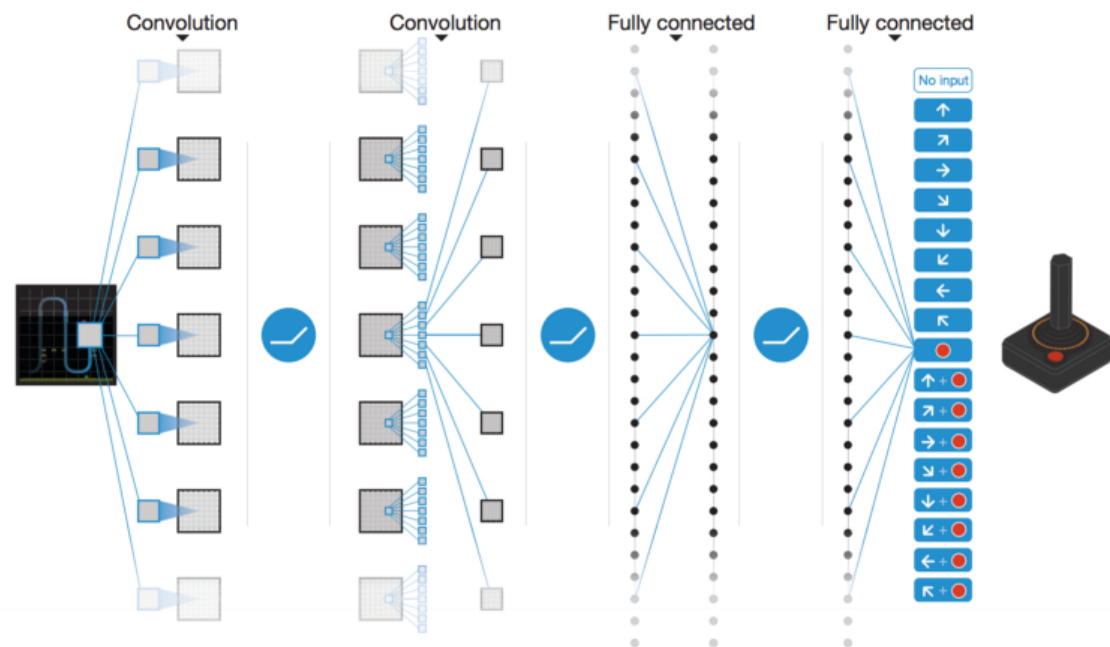
Input: the policy π to be evaluated
Initialize $V(s)$ arbitrarily (e.g., $V(s) = 0, \forall s \in \mathcal{S}^+$)
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 $A \leftarrow$ action given by π for S
 Take action A , observe R, S'
 $V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$
 $S \leftarrow S'$
 until S is terminal

8.3 SARSA와 Q학습 알고리즘 (1/3)

■ Bootstrapping Value Based RL

- Value function만을 학습
- Action의 선택은 greedy policy를 사용: $\pi_{k+1}(s) = \arg \max_a Q_k(s, a)$
- 대표적으로 SARSA와 Q-learning이 있음
- Table을 사용하는 방법과 Approximation을 사용하는 방법이 있음

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 400 & 0 \\ 0 & 0 & 0 & 320 & 0 & 500 \\ 0 & 0 & 0 & 320 & 0 & 0 \\ 0 & 400 & 256 & 0 & 400 & 0 \\ 320 & 0 & 0 & 320 & 0 & 500 \\ 0 & 400 & 0 & 0 & 400 & 500 \end{bmatrix} \end{matrix}$$



8.3 SARSA와 Q학습 알고리즘 (2/3)

■ SARSA

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

알고리즘 1: Sarsa 학습 알고리즘

$Q(s,a)$ 를 임의값으로 초기화

For 에피소드=1,..., n do

s 초기화

s 에 따르는 Q (예, ϵ -greedy)를 통해 유래된 a 를 선택

 For 시퀀스 $t=1, \dots, T$ do

a 선택, 보상 r 과 다음 상태 s' 관측

$s'=s_{t+1}$ 에 따르는 Q (예, ϵ -greedy)를 통해 유래된 $a'=a_{t+1}$ 을 선택

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

$s \leftarrow s'; a \leftarrow a';$

 End For

End For

- On-policy 알고리즘: 경험을 모으는 policy가 현재 policy임

8.3 SARSA와 Q학습 알고리즘 (3/3)

■ Q-learning

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

알고리즘 2; Q-Learning 알고리즘

$Q(s,a)$ 를 임의값으로 초기화

For 에피소드=1, ..., n do

s 초기화

 For 시퀀스 $t=1, \dots, T$ do

s 에 따르는 Q (예, ϵ -greedy)를 통해 유래된 a 를 선택

a 선택, 보상 r 과 다음 상태 $s'=s_{t+1}$ 관측

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

$s \leftarrow s'$;

 End For

End For

- Off-policy 알고리즘: 경험을 모으는 policy가 현재 policy와 다름

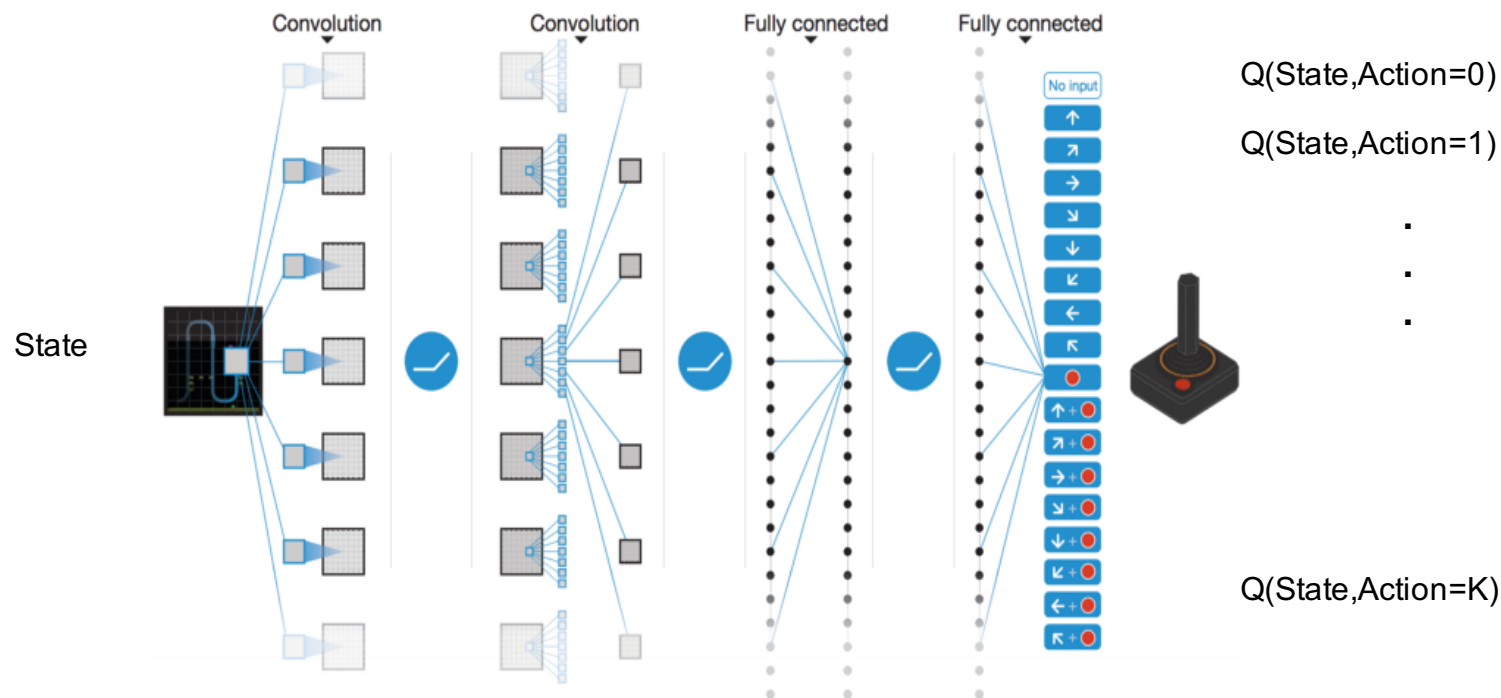
8.4 딥큐넷(DQN) (1/3)

- Q-learning의 업데이트 식

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

- DQN은 Q학습의 오류함수 최소화해 딥신경망과 오류역전파 알고리즘을 사용

$$L = \frac{1}{2} [r + \gamma \max_a Q(s', a) - Q(s, a)]^2$$



8.4 딥큐넷(DQN) (2/3)

■ Target Network Trick

- 학습 초기 $Q(s', a')$ 이 부정확하고 변화가 심함 → 학습 성능 저하
- DQN과 동일한 구조를 가지고 있으며 학습 도중 weight값이 변하지 않는 별도의 네트워크 (Target Network)에서 $Q(s', a')$ 를 계산 - Target Network의 weight값들은 주기적으로 DQN의 것을 복사

■ Replay Memory Trick

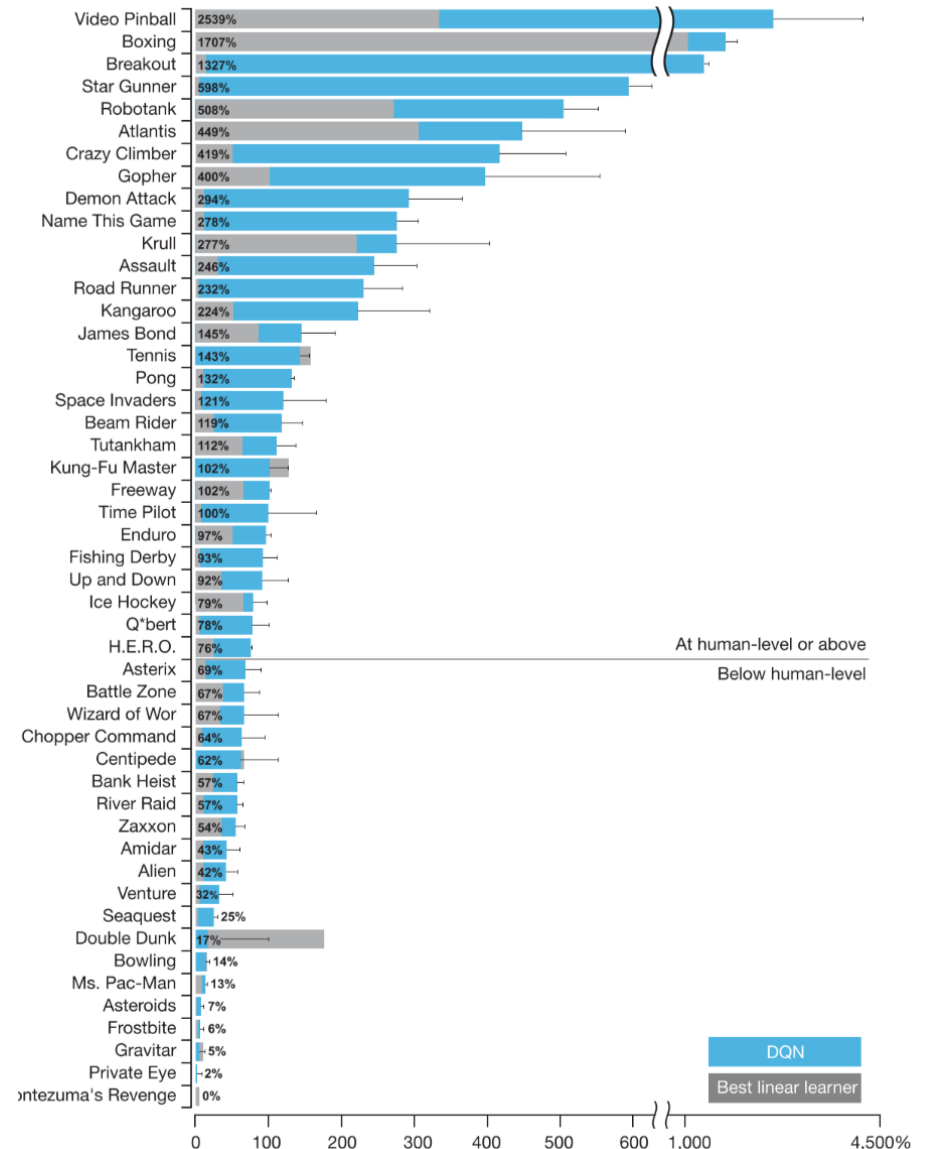
- Changing Data Distribution: Agent의 행동에 따라 들어오는 데이터의 분포가 변화함 (e.g. 어떤 mini batch를 학습한 후 무조건 왼쪽으로 가도록 policy가 변화 → 이후 왼쪽으로 가지 않는 경우의 데이터를 얻을 수 없게 되어 학습이 불가능)
- (State, Action, Reward, Next State) 데이터를 Buffer에 저장해 놓고 그 안에서 Random Sampling하여 mini batch를 구성하는 방법으로 해결

■ Reward Clipping Trick

- 도메인에 따라 Reward의 크기가 다르기 때문에 Q value의 크기도 다름
- Q value의 크기 variance가 매우 큰 경우 신경망 학습이 어려움
- Reward의 크기를 [-1, +1] 사이로 제한하여 안정적 학습

8.4 딥큐넷(DQN) (3/3)

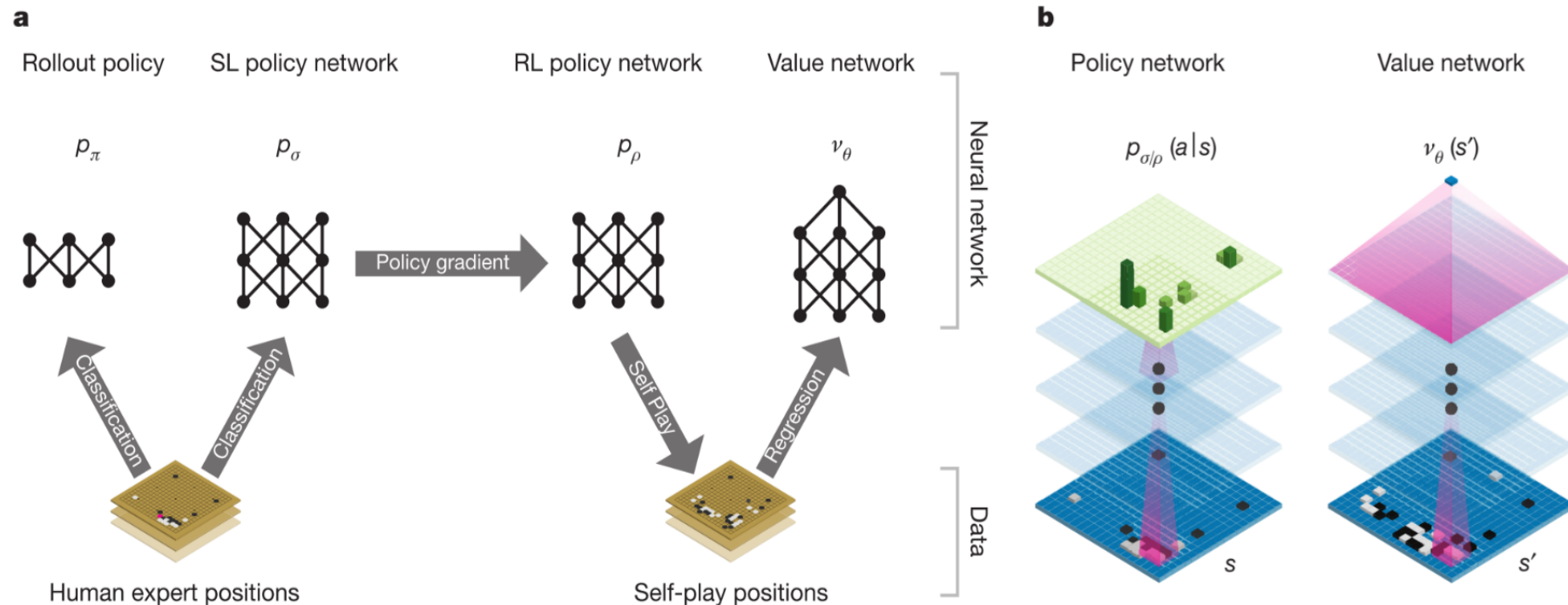
- Atari 2600 비디오 게임에서 실험
 - 절반 이상의 게임에서 사람보다 우수
 - 기존방식(linear)에 비해 월등한 향상
 - 일부 게임은 실패
(reward가 sparse, 복잡한 단계를 필요)
- Paper
Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." Nature 518.7540 (2015): 529-533.
 - Video
<https://www.youtube.com/watch?v=TmPfTpjtdgg>
 - Open source
https://github.com/gliese581gg/DQN_tensorflow



8.5 딥강화학습의 활용: AlphaGo (1/2)

■ AlphaGo

- 강화학습(A3C)과 몬테칼로 트리탐색(MCTS) 알고리즘을 결합
- (A3C = Asynchronous Actor-Critic Agents, MCTS = Monte-Carlo Tree Search)
- 탐색공간이 매우 큰 바둑에서 학습을 통해서 인간을 뛰어넘는 성능 데모
- 정책망(policy network, actor)과 가치망(value network, critic)을 따로 학습
- 정책망은 프로기사들의 기보를 사용해 지도학습 후 강화학습 적용



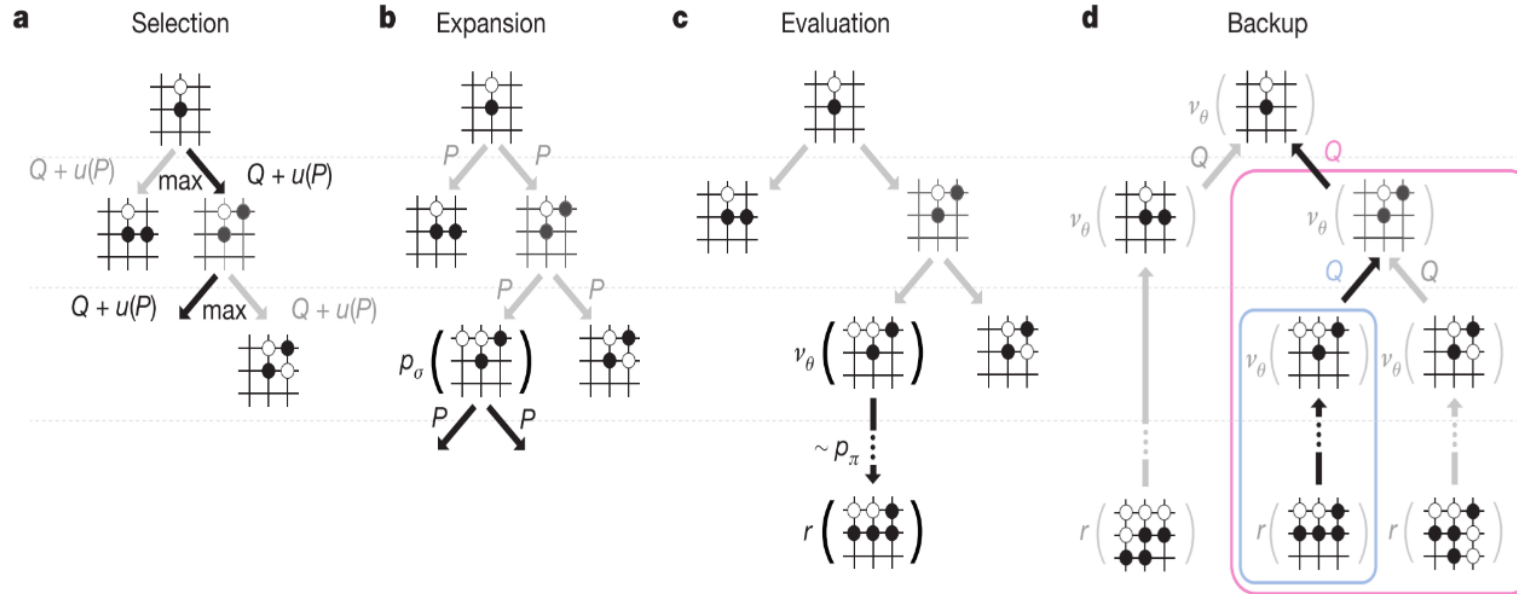
8.5 딥강화학습의 활용: AlphaGo (2/2)

■ Monte-Carlo Tree Search

$N(s,a)$ = 상태행동 방문횟수

- Selection: $a_t = \arg \max (Q(s_t, a) + u(s_t, a)), \quad u(s_t, a) \propto \frac{P(s_t, a)}{1 + N(s_t, a)}$
- Expansion: 앞 노드에 도착하면 정책망을 통해 새로운 앞 노드 생성
- Evaluation: $V(s_L) = (1 - \lambda)v_\theta(s_L) + \lambda z_L$
- Backup: $N(s, a) = \sum_i 1(s, a, i) \quad Q(s, a) = \frac{1}{N(s, a)} \sum_i 1(s, a, i) V(s_L^i)$

z_L = 시뮬레이션 통한 보상



Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., ... & Dieleman, S. (2016). Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587), 484-489.

요약

■ 강화학습과 MDP 문제

- MDP: State, Action, Reward, Transition, Discount로 구성. 순차적 의사결정
- 강화학습은 MDP 문제를 근사적으로 해결하는 머신러닝 방법
- Reward 기대치를 최대화하는 방향으로 Agent의 행동을 선택하도록 학습

■ Sarsa와 Q-Learning

- Value Function을 Approximation하고 이를 최대화하는 Action 선택
- Sarsa, Q-Learning
- DQN: Deep Neural Network로 Q값을 근사

■ 딥강화학습의 활용

- AlphaGo 바둑 AI
- RL과 Monte Carlo Tree Search
- 정책망, 가치망에 딥러닝을 활용

질문

- 감독학습, 무감독 학습과 비교하여 강화학습이 다른 점은 무엇인가?
- 마코프결정문제(MDP)를 정의하고 다양한 해결 방법들을 기술하시오.
 - 벨만 최적식, 동적프로그래밍의 개념을 설명하시오.
 - MDP 문제 해결을 위한 방법으로서의 강화학습을 설명하시오.
- 강화학습의 다양한 전략들을 기술하고 서로간의 차이를 설명하시오.
 - 모델기반 vs. 모델프리 RL의 차이를 설명하시오.
 - 오프라인 vs. 온라인 RL의 차이를 설명하시오.
 - On-policy vs. Off-policy RL의 차이를 설명하시오.
- MC학습(몬테칼로기반 동적프로그래밍 방식) 방법을 설명하시오.
- TD학습 방법을 기술하고 MC 방법과의 차이를 설명하시오.
- TD학습, Sarsa, Q학습 알고리즘을 설명하고 차이점을 기술하시오
- 알파고가 사용한 딥강화학습 DQN의 핵심 아이디어들을 설명하시오.